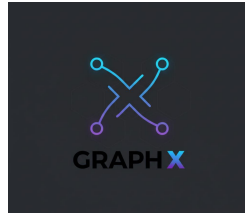

GraphX



ADAMYA VERMA	VANAMALA AMAN PREETHAM
KAUSHAL MORANKAR	CHIRAME TANISHQ ARAVIND
SAI DEEPAK SANA	KONIDALA ARJUN
BOLLA LOKESH REDDY	

Contents

1	Introduction	1
1.1	Background	1
1.2	Aim	2
2	Language Design	2
2.1	Data Types	2
2.2	Literals	2
2.3	Keywords	2
2.4	Operators	3
2.5	Comments	3
2.5.1	Single Line Comments	3
2.5.2	Multi-line Comments	3
2.6	Control Flow	4
2.6.1	If Statements	4
2.6.2	For Loops	4
2.6.3	While Loops	4
2.7	Functions	4
3	Graph Data Model	5
3.1	Structs	5
3.2	vertex_sets and edge_sets	5
3.3	Graph Type	5
3.4	Vectors	6
4	Graph Operations	6
4.1	Basic Operations	6
4.1.1	Adding and Removing Vertices/Edges	6
4.1.2	Graph Queries	7
4.2	VertexSets and EdgeSets Operations	7
4.3	Graph Operations	7
4.3.1	Union Operation	7
4.3.2	Intersection Operation	8
4.3.3	Difference Operation	8
4.3.4	Cartesian Product	8
4.3.5	BFS and DFS Traversals	8
5	Built-in Functions and Operators	8
5.1	Some Builtin Functions	9
5.2	Vector Operations	9
5.3	Set Operations	9
5.4	Graph Algorithms	10
6	Density-Aware Optimizations	10
7	Built-in Parameterised Algorithms	10
8	SoA (Structure of Arrays) Implementation	11
8.1	SoA vs. AoS: The Critical Difference	11
8.1.1	Why SoA Matters for Graph Algorithms	11
9	Input Format	12
9.1	Supported File Extensions	12

9.2	File Format Specifications	12
9.2.1	Edge List (.el)	12
9.2.2	Weighted Edge List (.wel)	12
9.2.3	Vertex List (.vl)	13
9.2.4	Graph File (.gr)	13
9.3	Output Format	13
10	SQL Like Queries	13
11	Optimized SSSP	14

Domain: Graph DSL

Types

- **Primitive Types:** `int`, `float`, `double`, `uint`, `bool`, `char`
- **Composite Types:** `vector`, `struct`, `string`
- **Graph Types:** `graph`, `vertex_set`, `edge_set`

Operators

- **Unary Operators:**
 - Metric closure of a graph
 - Transitive closure of a graph
 - Transpose of a graph
 - Complement of a graph
 - Connected components ($\#CC(G)$)
 - Power of a graph
- **Binary Operators:**
 - Union, Intersection, Difference, Cartesian Product
 - Tensor Product
 - `bfs/dfs` operators

Unique Selling Points (USP)

- Graph algebra
- Density-aware optimizations on dynamic graphs
- New SSSP (single-source shortest path) algorithm implementation
- Implementation of vertex cover and similar graph problems
- SQL-like declarative syntax for subgraph generation (eliminates explicit traversal)
- Structure of Arrays (SoA) based internal representation

1 Introduction

GraphX is a graph DSL. It treats graphs as abstract algebraic objects. It works on dynamic graphs and provides graph algebra, operators and functionalities for various purposes. It offers a purely algebraic interface where graphs can be constructed, transformed, composed, and optimised using high-level operators and functionalities. Adding some new functionalities from some recent progress, it covers wide variety of graphs and optimises based on whether the graph is dense or sparse.

1.1 Background

The use of graphs has grown immensely in recent years, driven by applications in social networks, recommendation systems, biological data analysis, and large-scale web search. Traditional programming languages require detailed implementations to manage data structures, parallelism, and scalability, which results in steep learning curves and error-prone development. This motivated us towards the need for a specialised yet intuitive domain-specific language (DSL) that can handle complex graph operations at scale, while remaining accessible to a broader range of users. Such a DSL would allow developers to express graph problems at a high level, with the system ensuring efficient execution.

1.2 Aim

The aim is to provide a powerful yet simple DSL for graph programming. With this DSL, users can construct and manipulate dynamic graphs with a variety of operations, including adding or removing nodes and edges, traversals, shortest path queries, connectivity checks, and subgraph extraction. In addition to these basic operations, the DSL supports advanced graph algorithms such as HLD, and minimum spanning tree. The DSL leverages SQL like declarative syntax for graph traversal and subgraph extraction without the need for manual traversal. It achieves scalable performance via SoA memory layout and SIMD-friendly execution.

2 Language Design

GraphX is designed with simplicity and performance in mind. The language provides high-level abstractions for graph operations while allowing for performance tuning through scheduling directives.

2.1 Data Types

GraphX provides several data types specifically designed for graph processing:

1. Primitive types

`int` Signed integer (typical 32-bit). Usage: counters, small ids (if not using large id space).

`uint` Unsigned integer (typical 32-bit). Usage: sizes, non-negative ids.

`float` 32-bit floating point. Usage: approximate weights, activations.

`double` 64-bit floating point. Usage: high-precision weights or distances.

`bool` Boolean values `true/false`. Usage: flags, filters.

`char` Single character (ASCII/UTF-8 code unit). Usage: compact labels.

2. Composite types

`vector`, `struct`, `string`

3. Graph types

`graph`, `vertex_set`, `edge_set`

2.2 Literals

GraphX supports standard literals for primitive types:

- **Integer literals:** `42`, `0xFF`, `0b1010`
- **Floating-point literals:** `3.14`, `6.022e23`
- **Boolean literals:** `true`, `false`
- **String literals:** `"Hello, GraphX!"`

2.3 Keywords

Table 1: Reserved Keywords in GraphDSL

<code>int</code>	<code>float</code>	<code>char</code>	<code>string</code>	<code>double</code>
<code>true</code>	<code>false</code>	<code>if</code>	<code>for</code>	<code>bool</code>
<code>while</code>	<code>break</code>	<code>continue</code>	<code>vector</code>	<code>struct</code>
<code>vertex_set</code>	<code>edge_set</code>	<code>graph</code>	<code>funk</code>	<code>-</code>
<code>const</code>	<code>var</code>	<code>new</code>	<code>-</code>	<code>-</code>

2.4 Operators

GraphX provides a rich set of operators for graph manipulation:

Table 2: GraphX Operators with Symbol Usage Examples

Operator	Symbol	Description	Example Usage
Metric Closure	<code>^@</code>	Computes shortest paths between all pairs of vertices	<code>G @</code>
Transitive Closure	<code>^*</code>	Computes reachability between vertices	<code>G ^*</code>
Transpose	<code>^T</code>	Reverses direction of all edges in a directed graph	<code>G ^T</code>
Complement	<code>~</code>	Computes graph complement (non-existing edges)	<code>~G</code>
<code>#CC(G)</code>	<code>#</code>	Number of connected components	<code>#G</code>
Power of Graph	<code>**</code>	Raises graph to power k	<code>G ** 2</code>
Union	<code>+</code>	Union of two graphs	<code>G1 + G2</code>
Intersection	<code>&</code>	Intersection of two graphs	<code>G1 & G2</code>
Difference	<code>-</code>	Graph difference	<code>G1 - G2</code>
Cartesian Product	<code>*</code>	Cartesian product of two graphs	<code>G1 * G2</code>
Tensor Product	<code>@</code>	Tensor (Kronecker) product of two graphs	<code>G1 @ G2</code>
BFS Operator	<code>bfs</code>	Breadth-First Search traversal from a node	<code>G.bfs(v)</code>
DFS Operator	<code>dfs</code>	Depth-First Search traversal from a node	<code>G.dfs(v)</code>

Precedence	Operator(s)	Associativity
1	<code>(...)</code> , <code>.bfs()</code> , <code>.dfs()</code>	n/a
2	<code>^T</code> , <code>^*</code> , <code>^@</code>	Left-to-right
3	<code>**</code>	Right-to-left
4	<code>~</code> , <code>#</code>	n/a
5	<code>@</code>	Left-to-right
6	<code>*</code>	Left-to-right
7	<code>&</code>	Left-to-right
8	<code>-</code>	Left-to-right
9	<code>+</code>	Left-to-right

Table 3: Operator precedence (highest → lowest)

2.5 Comments

2.5.1 Single Line Comments

Single Line Comments begin with `//` and extend to the end of the line.

```
1 // This is a single line comment
2 var x : int = 42; // This is also a comment
```

2.5.2 Multi-line Comments

Multi Line Comments start with `/*` and end with `*/`.

```

1  /*
2     This is a multi-line comment
3     that spans multiple lines
4  */
5  var y : int = 3.14;

```

2.6 Control Flow

2.6.1 If Statements

GraphX supports standard if-else statements:

```

1  if (x > 0) {
2      print("Positive");
3  } elif (x < 0) {
4      print("Negative");
5  } else {
6      print("Zero");
7  }

```

2.6.2 For Loops

GraphX provides for loops for iterating over ranges or collections:

```

1  // Iterating over a range
2  for i in range(0, 10, i+1) {
3      print(i); // Prints 0 through 9
4  }
5  // Dynamic step that depends on current iterator value
6  for i in range(0, 100, i + 10) {
7      print(i); // Prints 0, 10, 20, 30, 40, 50, 60, 70, 80, 90
8  }
9
10 // Iterating over a vertex set
11 for v in vertices {
12     print("Processing vertex: " + v);
13 }

```

Note: If no increment is provided in range function then it is treated as increment by 1.

2.6.3 While Loops

While loops continue execution as long as a condition is true:

```

1  var i : int = 0;
2  while (i < 10) {
3      print(i);
4      i = i + 1;
5  }

```

2.7 Functions

```

1  // Function with single return value (curly braces style)
2  fun square(x : int) : int {
3      return x * x;
4  }
5
6  // Function that returns the maximum of three values
7  fun max3(a : float, b : float, c : float) : float {
8      if (a >= b && a >= c) {
9          return a;

```

```

10 } elif (b >= a && b >= c) {
11 return b;
12 } else {
13 return c;
14 }
15 }

```

3 Graph Data Model

GraphX provides a rich data model for representing and manipulating graphs.

3.1 Structs

Structs define the properties of vertices and edges. Access vertex or edge data using flower brackets — for example: `age{person}`

```

1 // Define a vertex struct type
2 struct Person {
3     var name : string;
4     var age : int;
5 }
6
7 // Define an edge struct type with properties
8 struct Friendship {
9     var duration : int;
10    var strength : float;
11 }

```

3.2 vertex_sets and edge_sets

`vertex_sets` and `edge_sets` are the primary collections for working with graphs. Vertex set is the set of vertices. It can be declared by

```

1 // Create an edge set of Friendship structs between Person vertices
2 const socialNetwork : edge_set{Friendship}(Person, Person);
3
4 // Create a vertex set of Person structs
5 const people : vertex_set{Person} = socialNetwork.getvertices();
6
7 // Initialize with specific vertices
8 var activeUsers : vertex_set{Person} = new vertex_set{Person}(0);
9 activeUsers.add_vertex(5);
10 activeUsers.add_vertex(10);

```

3.3 Graph Type

GraphX introduces a first-class `graph` type that represents complete graph structures with vertices and edges, enabling powerful graph algebra operations.

```

1
2 // Create a graph using your existing vertex and edge sets
3 const people : vertex_set{Person} = new vertex_set{Person}(0);
4 const friendships : edge_set{Friendship}(Person, Person) = load("social_network.
5     el");
6 const socialGraph : graph{Person}(Friendship) = {
7     V = people,
8     E = friendships
9 };
10
11 // Or create directly from file
12 const socialGraph : graph{Person}(Friendship) = load("social_network.gr");

```



```

12
13
14 // REFERENCE (&): modifications affect original
15 const people& : vertex_set{Person} = socialGraph.getvertices();
16 const friendships& : edge_set{Friendship}(Person, Person) = socialGraph.getEdges
    ();
17
18 // Modifications immediately affect the original graph
19 people.add_vertex(100);
20 friendships.add_edge(1, 100, {strength=0.8});
21
22 // VALUE (no &): creates independent copy
23 const peopleCopy : vertex_set{Person} = socialGraph.getvertices();
24 const edgesCopy : edge_set{Friendship}(Person, Person) = socialGraph.getEdges();
25
26 // Modifications do NOT affect the original graph
27 peopleCopy.add_vertex(200);
28 edgesCopy.add_edge(1, 200, {strength=0.9});
29
30
31 // Get references (modifies original)
32 const vertices& : vertex_set{Person} = socialGraph.getvertices();
33 const edges& : edge_set{Friendship}(Person, Person) = socialGraph.getEdges();
34
35 // Get copies (independent)
36 const vertices : vertex_set{Person} = socialGraph.getvertices();
37 const edges : edge_set{Friendship}(Person, Person) = socialGraph.getEdges();

```

3.4 Vectors

Vectors allow efficient association of additional data with vertices or edges, supporting SoA-style operations for parallelism and memory efficiency.

```

1 // Vector for vertex properties
2 const age : vector{Person}(int) = 0;
3
4 // Vector for edge properties
5 const edgeWeight : vector{Friendship}(float) = 1.0;
6
7 // Accessing vector structs
8 fun updateAges() {
9     for v in people {
10         age[v] = age[v] + 1;
11     }
12 }

```

4 Graph Operations

GraphX provides a comprehensive set of operations for graph manipulation.

4.1 Basic Operations

4.1.1 Adding and Removing Vertices/Edges

```

1 // Add vertices
2 people.add_vertex(100);
3 people.add_vertex(101);
4
5 // Add edges
6 socialNetwork.add_edge(100, 101);
7 socialNetwork.add_edge(100, 101, {duration: 365, strength: 0.8});
8

```

```

9 // Remove vertices
10 people.remove_vertex(100);
11
12 // Remove edges
13 socialNetwork.remove_edge(100, 101);

```

Note: Vertex removal automatically updates all incident edges. It is not permitted to add an edge between vertices not present in vertex set. Only after adding the corresponding vertices in the vertex set, such an edge must be added.

4.1.2 Graph Queries

```

1 // Check if edge exists
2 var exists : bool = socialNetwork.isEdge(1, 2);
3
4 // Get number of vertices
5 var numVertices : int = people.size();
6
7 // Get number of edges
8 var numEdges : int = socialNetwork.size();
9
10 // Get degree of a vertex
11 var degree : int = socialNetwork.get_degree(5);
12
13 // Get indegree of a vertex
14 var degree : int = socialNetwork.get_indegree(5);
15
16 // Get outdegree of a vertex
17 var degree : int = socialNetwork.get_outdegree(5);
18
19 // Get neighbors of a vertex
20 var neighbors : vertex_set{Person} = socialNetwork.getNeighbors(5);

```

4.2 VertexSets and EdgeSets Operations

GraphX supports set operations on vertex sets and edge sets:

```

1 // Union of two vertex sets
2 var allUsers : vertex_set{Person} = activeUsers + inactiveUsers;
3
4 // Intersection of two vertex sets
5 var commonFriends : vertex_set{Person} = friendsOfA & friendsOfB;
6
7 // Difference of two vertex sets
8 var exclusiveFriends : vertex_set{Person} = friendsOfA - friendsOfB;

```

4.3 Graph Operations

GraphX supports a comprehensive set of graph algebra operations that follow mathematical notation for intuitive readability. These operations enable powerful graph transformations while maintaining consistency with established DSL patterns.

4.3.1 Union Operation

The union operation combines two graphs by merging their vertices and edges. It uses the + symbol for intuitive mathematical notation.

```

1 // Union of two graphs (combines edges and vertices)
2 var combinedNetwork : graph{Person, Friendship} = socialNetwork + purchaseNetwork
3 ;

```

```

4 // Union can be used in SQL-like queries
5 var result = (socialNetwork where V.age > 25) + (purchaseNetwork where V.income <
    50000);

```

4.3.2 Intersection Operation

The intersection operation retains only the common vertices and edges between two graphs. It uses the $\cap$ symbol for intuitive mathematical notation.

```

1 // Intersection of two graphs (retains common edges and vertices)
2 var commonConnections : graph{Person, Friendship} = socialNetwork &
    purchaseNetwork;
3
4 // Intersection in SQL-like queries
5 var result = (socialNetwork where V.age > 25) & (purchaseNetwork where V.income <
    50000);

```

4.3.3 Difference Operation

The difference operation removes vertices and edges from the first graph that are present in the second graph. It uses the $-$ symbol for intuitive mathematical notation.

```

1 // Difference between two graphs (removes edges/vertices from G1 present in G2)
2 var uniqueSocial : graph{Person, Friendship} = socialNetwork - purchaseNetwork;
3
4 // Difference in SQL-like queries
5 var result = (socialNetwork where V.age > 25) - (purchaseNetwork where V.income <
    50000);

```

4.3.4 Cartesian Product

The Cartesian product creates a new graph where vertices are pairs of vertices from the input graphs, and edges connect compatible pairs. It uses the $\times$ symbol for intuitive mathematical notation.

```

1 // Cartesian product of two graphs (creates composite vertices and edges)
2 var networkProduct : graph{Person Person, Friendship Friendship} =
3 socialNetwork * purchaseNetwork;
4
5 // Cartesian product in SQL-like queries
6 var result = (socialNetwork where V.age > 25) * (purchaseNetwork where V.income <
    50000);

```

4.3.5 BFS and DFS Traversals

GraphX provides built-in traversal operators for Breadth-First Search and Depth-First Search with intuitive method syntax.

```

1 // BFS traversal starting from node v
2 var bfsResult : vertex_set{Person} = socialNetwork.bfs(1);
3
4 // DFS traversal starting from node v
5 var dfsResult : vertex_set{Person} = socialNetwork.dfs(1);

```

Note: All our unary and binary operators follow the same syntax rules as the examples shown above. Since every operator listed in the operator table adheres to this unified syntax, we abstract away the repetitive details and encourage the user to explore them independently.

5 Built-in Functions and Operators

GraphX provides a rich set of built-in functions for graph processing.

5.1 Some Builtin Functions

5.2 Vector Operations

Table 4: Vector Operations

Function	Description
<code>.add_edge(type,type,weight (optional))</code>	Add the edge in the edge_set
<code>.remove_edge(type,type)</code>	Removes the edge in the edge_set
<code>.add_vertex(type)</code>	Add vertex in the vertex_set
<code>.remove_vertex(type)</code>	Remove the vertex in the vertex_set
<code>.update_edge(type,type,weight)</code>	Update the edge weight
<code>.get_vertices()</code>	Get the vertex set from the graph
<code>.get_edges()</code>	Get the edge set for the graph
<code>.is_edge(type,type)</code>	Checks if there is an edge between two vertices

5.3 Set Operations

Table 5: Set Operations

<code>getvertices()</code>	Get all vertices in the graph	<code>vertices = edges.getvertices()</code>
<code>get_outdegree(k)</code>	Get out-degree of a vertex	<code>degrees = edges.get_outdegree(k)</code>
<code>get_indegree(k)</code>	Get in-degree of a vertex	<code>degrees = edges.get_indegree(k)</code>
<code>get_degree(k)</code>	Get degree of a vertex	<code>degrees = edges.get_degree(k)</code>
<code>transpose()</code>	Create a transposed graph	<code>transposed = edges.transpose()</code>
<code>get_neighbors(v)</code>	Get neighbors of vertex v	<code>neighbors = edges.get_neighbors(5)</code>
<code>get_inneighbors(v)</code>	Get incoming neighbors	<code>inNeighbors = edges.get_inneighbors(5)</code>
<code>get_outneighbors(v)</code>	Get outgoing neighbors	<code>outNeighbors = edges.get_outneighbors(5)</code>

5.4 Graph Algorithms

Table 6: Graph Algorithms

Function	Description	Example Usage
<code>sssp(startVertex)</code>	Single-source shortest path (optimized)	<code>distances = socialNetwork.sssp(startVertex)</code>
<code>dij(startVertex)</code>	Single-source shortest path (Dijkstra’s algorithm)	<code>distances = socialNetwork.dij(startVertex)</code>
<code>mst()</code>	Minimum spanning tree of the graph	<code>mstEdges = socialNetwork.mst()</code>
<code>blfr(startVertex)</code>	Bellman–Ford shortest path algorithm	<code>distances = socialNetwork.blfr(startVertex)</code>
<code>diam()</code>	Diameter of the graph	<code>diameter = socialNetwork.diam()</code>
<code>topo()</code>	Topological sorting of the graph	<code>topo_order = socialNetwork.topo()</code>
<code>bridges()</code>	Bridges detection in a graph	<code>bridge_vector = socialNetwork.bridges()</code>
<code>atp()</code>	Articulation points in a graph	<code>articulation_vector = socialNetwork.atp()</code>
<code>cg()</code>	Gives centroid of the graph	<code>centroid = socialNetwork.cg()</code>
<code>epen()</code>	Finding whether euler path exists or not	<code>euler_path = socialNetwork.epen()</code>
<code>is_cycle()</code>	cycle detection in a graph	<code>has_cycle = socialNetwork.is_cycle()</code>
<code>bpm()</code>	Bipartite matching in a graph	<code>matching_vector = socialNetwork.bpm()</code>
<code>vc(k)</code>	vertex cover of a graph of size k	<code>vertex_cover_vector = socialNetwork.vc(k)</code>

6 Density-Aware Optimizations

GraphX implements sophisticated density-aware optimizations that automatically select the most efficient data structures and algorithms based on the characteristics of the input graph. Unlike traditional graph processing frameworks that require manual configuration, GraphX’s compiler analyzes graph density at runtime and makes intelligent decisions without requiring user intervention. For graph storage, GraphX dynamically selects between:

- Adjacency list representation for sparse graphs (density < 0.01)
- Adjacency matrix for dense graphs (density > 0.5)
- Hybrid approach for medium-density graphs ($0.01 \leq \text{density} \leq 0.5$)

Certain graph algorithms mentioned above use density of the graphs for optimizations.

7 Built-in Parameterised Algorithms

In GraphX, we incorporate parameterized algorithms for certain graph-related problems such as vertex cover. These are included as built-in functions to provide fixed-parameter tractable (FPT) time solutions. Specifically, such algorithms run in time of the form $O(f(k) \cdot n^{O(1)})$, where k is the chosen

parameter and n is the input size. By leveraging this paradigm, our language ensures efficient handling of computationally hard problems when the parameter is small, thereby extending the usability of the framework to more complex graph scenarios.

8 SoA (Structure of Arrays) Implementation

The Structure of Arrays (SoA) pattern is a fundamental design decision that enables GraphX to achieve high performance in graph processing. Unlike traditional object-oriented approaches that use Array of Structures (AoS), GraphX stores properties of graph elements in separate contiguous arrays, providing significant performance benefits for graph algorithms.

8.1 SoA vs. AoS: The Critical Difference

The key distinction between these approaches lies in how data is organized in memory:

- **Array of Structures (AoS):** Data for the same entity is stored contiguously

```
1 // AoS pattern (traditional object-oriented approach)
2 struct Person {
3     int age;
4     string name;
5     vector<int> friends;
6 };
7 vector<Person> people; // [ {25, "Alice", [1,2,3]}, {32, "Bob", [0,2]}, ...
                        ]
```

- **Structure of Arrays (SoA):** Data for the same property is stored contiguously

```
1 // SoA pattern (used by GraphX)
2 vector<int> age; // [25, 32, 41, ...] (all ages together)
3 vector<string> name; // ["Alice", "Bob", ...] (all names together)
4 vector<uint64_t> row_ptrs; // CSR format
5 vector<uint32_t> col_indices; // CSR format
```

8.1.1 Why SoA Matters for Graph Algorithms

Graph algorithms typically process a single property across many vertices (e.g., updating distances in BFS). The SoA pattern provides critical performance advantages:

- **Better cache locality:** When processing one property across many vertices, the CPU can load contiguous blocks of memory containing just that property.
- **Vectorization:** Modern CPUs can process multiple values in parallel when they're stored contiguously.
- **Memory efficiency:** Only allocate memory for properties you actually use.
- **Algorithm specialization:** Different algorithms can use different subsets of properties.

For example, in Breadth-First Search (BFS), when processing a vertex's neighbors, the algorithm frequently checks if each neighbor has been visited by examining the **distance** array. With SoA, the CPU can efficiently process the **distance** array sequentially when checking multiple neighbors, resulting in significantly fewer cache misses compared to AoS where each cache line would contain mixed data (distance, parent, age, etc.). This is particularly important in graphs where vertices often have high degrees, requiring many distance checks per processed vertex.

Property Access Comparison:

SoA: `age[v] = 25;` (direct array access)

AoS: `people[v].age = 25;` (struct dereferencing)

9 Input Format

GraphX works on a file-based approach for graph input, where users provide graph data through standardized file extensions that explicitly indicate the content type and format. This design makes graph data loading intuitive and self-documenting.

9.1 Supported File Extensions

Extension	Content Type	Description
.el	Edge List	Simple edge list (source, destination) pairs
.wel	Weighted Edge List	Edge list with weights (source, destination, weight)
.vl	Vertex List	Vertex list with optional properties
.gr	Graph File	Complete graph description (vertices + edges)

Table 7: GraphX Supported File Extensions

9.2 File Format Specifications

9.2.1 Edge List (.el)

Contains unweighted edges in simple text format:

```
# Comments start with #
# Format: source_id destination_id
0 1
0 2
1 2
2 3
3 4
```

Usage in GraphX:

```
1 const friendships : edge_set{Friendship}(Person, Person) = load("social_network.
   el");
2 const socialGraph : graph{Person}(Friendship) = load("social_network.el");
```

9.2.2 Weighted Edge List (.wel)

Contains weighted edges:

```
# Comments start with #
# Format: source_id destination_id weight
0 1 0.8
0 2 0.5
1 2 0.9
2 3 0.7
3 4 0.6
```

Usage in GraphX:

```
1 const weightedEdges : edge_set{WeightedFriendship}(Person, Person) = load("
   social_network.wel");
2 const weightedGraph : graph{Person}(WeightedFriendship) = load("social_network.
   wel");
```

9.2.3 Vertex List (.vl)

Contains vertex information with optional properties:

```
# Comments start with #
# Format: vertex_id [property1] [property2] ...
0 "Alice" 25
1 "Bob" 30
2 "Charlie" 22
3 "David" 28
4 "Eve" 26
```

Usage in GraphX:

```
1 const people : vertex_set{Person} = load("people.vl");
2 const names : vector{Person}(string) = load("people.vl", column=1);
3 const ages : vector{Person}(int) = load("people.vl", column=2);
```

9.2.4 Graph File (.gr)

Contains complete graph description in JSON-like format:

```
{
  "vertices": [
    {"id": 0, "name": "Alice", "age": 25},
    {"id": 1, "name": "Bob", "age": 30},
    {"id": 2, "name": "Charlie", "age": 22}
  ],
  "edges": [
    {"source": 0, "destination": 1, "weight": 0.8, "duration": 5},
    {"source": 0, "destination": 2, "weight": 0.5, "duration": 3},
    {"source": 1, "destination": 2, "weight": 0.9, "duration": 7}
  ]
}
```

Usage in GraphX:

```
1 const completeGraph : graph{Person}(Friendship) = load("social_network.gr");
2 const (vertices, edges) = loadGraph("social_network.gr"); // Returns both
   components
```

9.3 Output Format

GraphX can also export graphs to these same formats:

```
1 // Export to different formats
2 export(socialGraph, "output_network.el"); // Edge list
3 export(socialGraph, "output_network.wel"); // Weighted edge list
4 export(socialGraph, "output_network.gr"); // Complete graph file
5
6 // Export with custom formatting
7 export(socialGraph, "output_network.el",
8       format={source_column=0, destination_column=1, separator="\t"});
```

10 SQL Like Queries

GraphX embeds a declarative, SQL-inspired sub-language that turns subgraph extraction into an intuitive, predicate-driven operation. Users phrase filters as Boolean expressions over vertex or edge

properties; the system rewrites these predicates into optimised graph traversals, automatically selects dense/sparse representations, and returns a new graph object containing only the vertices and edges that satisfy the condition. This yields a concise, analytics-ready subgraph without exposing implementation details or scheduling decisions.

11 Optimized SSSP

GraphX incorporates the latest deterministic SSSP breakthrough from Tsinghua University, the first algorithm to break the half-century-old $O(m + n \log n)$ barrier for sparse, real-weighted directed graphs. The solver runs in $O(m \log^{2/3} n)$ time—an asymptotic improvement that translates to 20–50% faster execution on million-node sparse instances—by abandoning global priority-queue sorting in favour of clustering-based selective scouting and on-demand edge relaxation. Internally, the engine interleaves this $\log^{2/3} n$ routine with the standard $O(m + n \log n)$ Dijkstra kernel, automatically selecting the faster path for the detected density regime, while exposing the same declarative SSSP interface to users.